Attention and Transformers

ECON 8310: Business Forecasting — Lecture 9, Part 2

Department of Economics

University of Nebraska at Omaha

August 25, 2026

Lecture Outline

What Recurrence Could Not Do

Two constraints that are built into reading a sequence one step at a time.

Two Problems Built Into Recurrence

Part 1 ended with a working recurrent forecaster. Both of its remaining problems come from the same source — it reads strictly left to right.

The information bottleneck. Everything the model knows about step 3 must survive being squeezed through every hidden state between 3 and T. An LSTM's gates make that survival more likely, but the whole history is still compressed into one fixed-size vector.

The sequential bottleneck. Step t cannot be computed until step $t-1$ is finished. That is a hard constraint: a 1,000-step sequence takes 1,000 sequential operations no matter how many GPUs you own. Convolutions parallelize; recurrence does not.

Attention removes both at once. Let every position look directly at every other position, in one operation. No compression through intermediate states, and no ordering constraint on the computation.

The Attention Mechanism

A learned, weighted lookup — which is a more useful way to read it than the formula suggests.

Scaled Dot-Product Attention (Vaswani et al. 2017)

The whole mechanism

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Q = queries, K = keys, V = values, d_k = key dimension.

Read it as a **lookup table with soft matching**. Each position emits a **query**: *"what am I looking for?"* Every position also offers a **key**: *"here is what I have"* — and a **value**: *"here is what I would contribute."*

$QK^\top$ scores every query against every key. The softmax turns those scores into weights that sum to one. The output is a weighted average of the values, with more weight where query and key matched.

Why divide by $\sqrt{d_k}$? Dot products grow with dimension, and large inputs push the softmax toward one-hot, where its gradient nearly vanishes. The scaling keeps it trainable.

What That Buys a Forecaster

Concretely: predicting units for the first week of December, the model can place high weight on *last* December and on the surrounding weeks, and near-zero weight on an unremarkable Tuesday in March.

Two properties make that different from anything so far.

The weights are learned, not specified. You do not tell the model that lag 52 matters. Compare that with LASSO in Lecture 6, where you built `lag52` by hand and the penalty decided whether to keep it.

The path is direct. Attention reaches week $t - 52$ in *one* operation, where a recurrent model needs 52 successive state updates and a CNN needs a receptive field that wide. Distance costs attention nothing.

> The price is quadratic: scoring every position against every other is $O(T^2)$ in time and memory — fine at $T = 26$, painful at $T = 10{,}000$.

Multi-Head Attention

One attention operation produces one set of weights — one notion of what is relevant. That is limiting, because different kinds of relevance coexist.

Multi-head attention runs h attention operations in parallel on different learned projections of the same input, then concatenates:

$$\text{MultiHead}(Q, K, V) = \text{concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O$$

Each head can specialize. In a forecasting model one head might track weekly position, another the annual cycle, another the effect of promotional flags — all attending over the same window at once.

The cost is not what you would guess. Each head works in a lower-dimensional subspace of size d_{model}/h, so eight heads cost about the same as one head at full width. You buy the diversity nearly for free.

Our model below uses `nhead=4` with `d_model=64` — four heads of 16 dimensions each.

The Transformer Encoder

Attention, plus the three pieces that make it trainable.

The Encoder Block

A Transformer encoder layer is four components in a fixed arrangement.

Multi-head attention	Every position attends to every position.
Residual + Layer-Norm	$\mathbf{z} = \text{LayerNorm}(\mathbf{x} + \text{MultiHead}(\mathbf{x}))$. The skip connection from Lecture 8, doing the same job.
Feed-forward sublayer	A small MLP applied to each position independently — the Lecture 7 network, at every step.
Residual + Layer-Norm	Again, around the feed-forward block.

Stack N of those and you have the encoder. Nothing in the block is new to you except attention itself — the residual connections are ResNet's, and the feed-forward sublayer is the network from Lecture 7.

One thing is conspicuously missing, and it is not an oversight. Nothing above depends on the *order* of the input.

Positional Encoding: Attention Cannot Tell Time

Attention computes a weighted average over positions. Averaging does not care about order — shuffle the input sequence and every attention output is unchanged.

Attention is permutation-invariant. For language that is fatal, and for forecasting it is worse: a model that cannot distinguish last week from thirty weeks ago is not forecasting at all.

The fix is to add position information to the input itself, before attention sees it. The original paper uses fixed sinusoids of different frequencies:

$$PE_{(t,2i)} = \sin\left(\frac{t}{10000^{2i/d}}\right), \qquad PE_{(t,2i+1)} = \cos\left(\frac{t}{10000^{2i/d}}\right)$$

Each position gets a distinct pattern across dimensions, and nearby positions get similar patterns — so the model can learn relative distance, not just absolute index.

We measure what removing it costs on the next slide but one.

Why Transformers Took Over

Three properties, in the order they mattered commercially.

Parallelism. Every position is computed at once, so training scales across hardware in a way recurrence never could. This, more than accuracy, is why the architecture won — it could absorb far more data and compute per unit of wall-clock time.

Direct long-range paths. Any position reaches any other in one step, so there is no distance at which information degrades.

Scale. The architecture kept improving as parameters and data grew, well past the point where earlier architectures plateaued. BERT (Devlin et al. 2019) is an encoder stack used for understanding; GPT is a decoder stack used for generation. Both are this block, repeated.

> Every one of those advantages is about **large data and long sequences**. Hold that thought against a 26-week window and 5,940 training examples.

Transformers for Time Series

The implementation, the measurement, and an honest reading of it.

Transformer Encoder in PyTorch

Minimal forecasting encoder

```
layer = nn.TransformerEncoderLayer(d_model=64, nhead=4,
dim_feedforward=128, dropout=0.1, batch_first=True)
self.enc = nn.TransformerEncoder(layer, num_layers=2)
self.inp = nn.Linear(n_features, 64)
self.fc  = nn.Linear(64, 1)

def forward(self, x):
h = self.inp(x) + self.pe     # position must be added
return self.fc(self.enc(h)[:, -1, :]).squeeze(-1)
```

`nn.TransformerEncoderLayer` gives you the whole block — but not the positional encoding. You add that yourself, and forgetting is a silent failure.

Multi-step forecasting also needs a causal mask.

Does It Work?

Same weekly panel, same 26-week windows, same test block as everything since Homework 3.

Model	Test RMSE	Parameters
LASSO, 46 engineered features	**744**	46
XGBoost	781	—
Vanilla RNN	**842**	4,545
Transformer encoder, 2 layers	**990**	67,329
Random forest	899	—
LSTM	987	17,985
1D CNN, best variant	1,202	8,161
Seasonal naive	2,152	0

The Transformer beats the CNN and **ties the LSTM** — 990 against 998, a difference smaller than the run-to-run spread. It loses to a vanilla RNN holding **one-fifteenth** as many parameters, and to a 46-coefficient linear model.

Reading That Result Honestly

Every advantage on the "why Transformers took over" slide is an advantage *at scale*, and this problem has none of the scale.

Parallel computation	Real, but irrelevant — the RNN trains in seconds either way.
Direct long-range paths	Worth little when the signal is dominated by recent weeks.
Scales with data	5,940 training windows is not the regime where that pays.

And positional encoding is doing real work, which we can measure. Remove it and the same model must forecast from an unordered bag of 26 weeks:

Transformer **with** positional encoding	**990**
Transformer **without** positional encoding	1,282

292 RMSE — dropping it below even the 1D CNN. That gap is the price of permutation invariance, and the reason the sinusoids exist.

Key Takeaways

Six architectures, one series, and what the ranking actually taught us.

Sequence Architecture Comparison

Architecture	Long range	Parallel	Needs
1D CNN	receptive field only	yes	modest data
Vanilla RNN	$\sim$10–30 steps	no	modest data
LSTM	hundreds of steps	no	more data
Transformer	unlimited, $O(T^2)$	**yes**	**much more data**

Choose by the structure of your problem, not by recency of the architecture. Short window and recent-lag signal: a plain RNN, or honestly a linear model. Long sequences, abundant data, dependencies at arbitrary distance: a Transformer, and nothing else comes close.

That second case is language, which is why this architecture reshaped that field and has been slower to displace gradient boosting in tabular forecasting — where the M5 competition was won by ensembles of exactly the methods in Lectures 5 and 6.

Lecture 9 Part 2: Key Takeaways

1. **Attention** is a learned soft lookup: score every query against every key, softmax the scores, average the values. Distance in the sequence costs nothing.
2. The price is **quadratic** in sequence length — fine at 26 steps, the central constraint at 10,000.
3. **Multi-head** attention runs several notions of relevance in parallel, at roughly the cost of one, by splitting the dimension.
4. An encoder block is attention + **residual** + LayerNorm + a feed-forward sublayer. Only attention is new — the residuals are ResNet, the sublayer is Lecture 7.
5. **Attention is permutation-invariant**, so position must be added explicitly. PyTorch does not do this for you, and omitting it fails silently.
6. On our panel the Transformer beat the LSTM and CNN, and **lost to a vanilla RNN with one-fifteenth the parameters**. Its advantages are advantages at scale.

Next: Bayesian Statistics I — a different answer to the question of what a forecast should tell you.

References I

- Devlin, Jacob et al. (2019). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, pp. 4171–4186.
- Vaswani, Ashish et al. (2017). "Attention Is All You Need". In: *Advances in Neural Information Processing Systems*. Vol. 30, pp. 5998–6008.